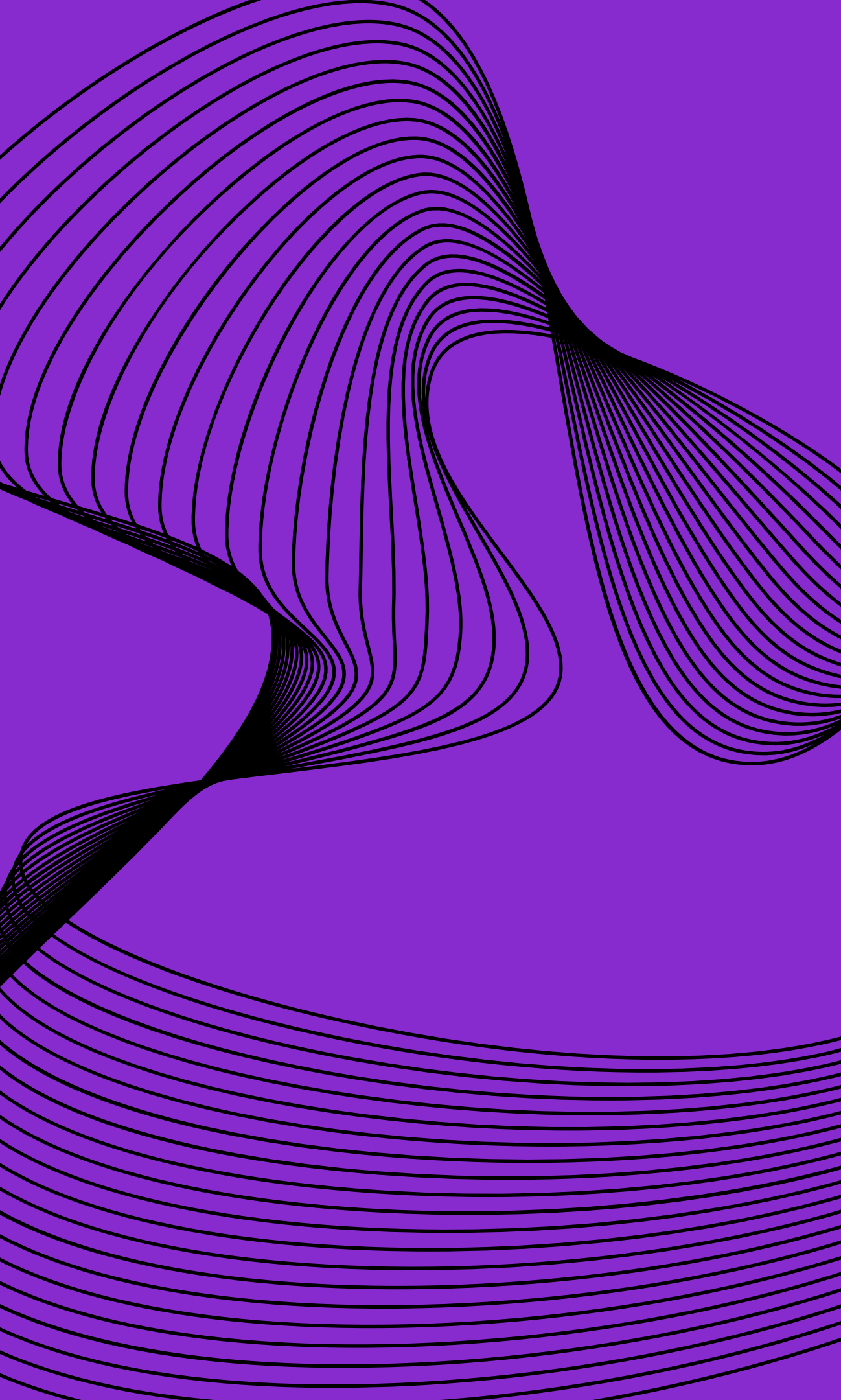
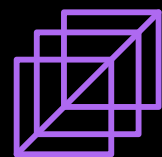




期末專題

數獨



邱岱瑋

鍾錦沛

期中進度



單一棋盤

只有一種棋盤

基礎數獨

只有一種難度

判斷對錯

已有一個固定的答案，
根據輸入值與正確答案
判斷對錯

三次錯誤限制

避免玩家隨意猜數字

新增功能



隨機棋盤

每次生成的都是不同的
棋盤

四種難度

- 簡單
- 普通
- 困難
- 極限

提示功能

給予三次提示機會

計時功能

增加娛樂性、刺激性

[37108] 數獨3.0.exe

週期事件 執行緒:

C:\Users\User\source\repos\數

+

▼

===== 隨機生成數獨挑戰賽 =====

請選擇遊戲難度：

[1] 簡單 (挖空 30 格, 容錯 3 次, 提示 3 次, 不限時)

[2] 普通 (挖空 42 格, 容錯 3 次, 提示 3 次, 不限時)

[3] 困難 (挖空 54 格, 容錯 3 次, 提示 3 次, 不限時)

[4] 極限 (挖空 54 格, 容錯 3 次, 無提示, 限時 5 分鐘)

請輸入選擇 (1-4): |

===== 隨機生成數獨挑戰賽 =====
目前錯誤次數：0 / 3

[計時模式] 已遊玩時間：0 分 7 秒
[提示] 剩餘提示機會：3 / 3 (輸入 0 0 0 取得提示)

	1	2	3	4	5	6	7	8	9
1	.	.	.	.	1	.	5	9	3
2	5	9	3	7	.	8	.	.	.
3	2	.	1	.	5	.	.	.	.
4	.	7	.	.	.	.	9	.	.
5	.	.	.	8	.	3	7	.	.
6	.	5	9	4	7	2	.	.	.
7	.	2	5	.	3	1	6	8	7
8	8	.	7	.	.	.	.	5	9
9	9	3	.	5	.	7	.	4	2

請輸入作答 [列] [行] [數字] (例如：1 3 4):

===== 隨機生成數獨挑戰賽 =====
目前錯誤次數：0 / 3

[極限挑戰] 剩餘時間：4 分 58 秒
[提示] 極限模式不提供提示功能！

	1	2	3	4	5	6	7	8	9
1	.	.	.	.	.	.	.	.	.
2	.	.	6	4	.	7	.	.	.
3	.	2	1	.	.	.	.	.	.
4	.	.	.	.	.	.	9	.	.
5	2	9	.	.	.	8	.	.	1
6	1	6	7	.	4	5	2	.	3
7	.	.	.	.	.	.	.	.	.
8	9	4	5	.	6	.	.	.	7
9	7	1	3	8	.	.	6	.	.

請輸入作答 [列] [行] [數字] (例如：1 3 4): |

===== 遊戲結束 =====

	1	2	3	4	5	6	7	8	9
1	1	4	8	5	2	3	7	9	6
2	3	9	5	4	7	6	1	8	2
3	7	2	6	8	9	1	3	4	5
4	2	5	4	3	6	7	9	1	8
5	8	7	9	1	4	5	2	6	3
6	6	1	3	2	8	9	5	7	4
7	5	8	7	6	1	2	4	3	9
8	4	3	1	9	5	8	6	2	7
9	9	6	2	7	3	4	8	5	1

恭喜你完成了這則隨機產生的數獨！太強了！
總共花費時間：4 分 59 秒

===== 遊戲結束 =====

	1	2	3	4	5	6	7	8	9
1	1	.	.	.	2	6	8	.	7
2	.	.	.	.	7	1	.	.	.
3	.	.	.	.	3	.	4	.	.
4	3	1	6	.	4	5	.	.	.
5	.	5	8	.	1	.	.	.	2
6	9	.	.	.	6	.	.	.	.
7	5	.	.	6	8	3	1	2	4
8	.	.	.	1	5	4	7	8	9
9	8	4	1	7	9	2	3	5	6

很遺憾，時間到！挑戰失敗。

隨機棋盤

對角線宮格互不干涉的特性，隨機填入數字 (fillDiagonal)，這保證了題目的隨機多樣性；接著透過遞迴回溯演算法(fillRemaining)，自動求解出一個合法的 81 格解答並存為後台標準答案。

遞迴回溯演算法

1. 電腦從左上角第一格開始，試著填入數字 1。
2. 檢查 1. 是否安全（符合行、列、宮格不重複的規則）。如果安全，電腦就前進到下一格。
3. 下一格電腦也試著填數字。如果一路順利，就一直填下去。
4. 遇到死路：假設填到第 15 格時，電腦發現 1 到 9 不管填哪一個，都會跟周圍的數字衝突。
5. 回溯：電腦會立刻清空這一格，退回第 14 格，把第 14 格原本填的數字（假設是 3）擦掉，改填 4，然後再次嘗試往下走。
6. 電腦不斷「嘗試、退回、改填」，直到 81 格全部填滿為止。

```

void Sudoku::fillDiagonal() {
    for (int i = 0; i < SIZE; i = i + 3) {
        fillBox(i, i);
    }
}

```

	1	2	3	4	5	6	7	8	9
1	.	.	.	.	.	3	.	.	7
2	7	.	.	6	.	.	.	.	.
3	.	3	.	7	.	.	1	6	9
4	.	.	3	.	.	.	7	.	.
5	8	9	.	2	.	.	.	.	.
6	.	.	.	.	1	9	.	.	8
7	4	1	8	.	.	.	2	.	.
8	.	.	.	8	7	.	9	.	.
9	.	7	.	3	.	.	8	.	.

```

bool Sudoku::fillRemaining(int i, int j) {
    if (j >= SIZE && i < SIZE - 1) {
        i = i + 1;
        j = 0;
    }
    if (i >= SIZE && j >= SIZE) return true;
    if (i < 3) {
        if (j < 3) j = 3;
    }
    else if (i < SIZE - 3) {
        if (j == (int)(i / 3) * 3) j = j + 3;
    }
    else {
        if (j == SIZE - 3) {
            i = i + 1;
            j = 0;
            if (i >= SIZE) return true;
        }
    }
    for (int num = 1; num <= SIZE; num++) {
        if (isSafe(i, j, num)) {
            grid[i][j] = num;
            if (fillRemaining(i, j + 1)) return true;
            grid[i][j] = 0;
        }
    }
    return false;
}

```

難度選擇

簡單(挖空 30 格, 容錯 3 次, 提示 3 次, 不限時)

普通(挖空 42 格, 容錯 3 次, 提示 3 次, 不限時)

困難(挖空 54 格, 容錯 3 次, 提示 3 次, 不限時)

極限(挖空 54 格, 容錯 3 次, 無提示, 限時 5 分鐘)

```
void Sudoku::removeKDigits(int k) {  
    int count = k;  
    while (count != 0) {  
        int cellId = getManualRand(SIZE * SIZE);  
        int i = cellId / SIZE;  
        int j = cellId % SIZE;  
        if (grid[i][j] != 0) {  
            count--;  
            grid[i][j] = 0;  
        }  
    }  
}
```

提示功能

程式不是固定提示某個位置，而是每一次呼叫時，都會即時重新掃描當前盤面，只針對目前還是空格的地方進行隨機挑選，避免重複提示。

我們將000定義為提示暗號。因為正常的數獨座標是1~9，輸入0是不合法的，因此將其作為提示非常安全且不會與一般座標衝突。

```

bool Sudoku::getHint(int& outR, int& outC, int& outVal) {
    int emptyCount = 0;
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (grid[i][j] == 0) emptyCount++;
        }
    }
    if (emptyCount == 0) return false;

    int target = getManualRand(emptyCount);
    int current = 0;
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (grid[i][j] == 0) {
                if (current == target) {
                    grid[i][j] = solution[i][j];
                    outR = i;
                    outC = j;
                    outVal = solution[i][j];
                    return true;
                }
                current++;
            }
        }
    }
    return false;
}

```

計時功能

程式不採用傳統會阻斷執行、讓畫面暫停的標準輸入，而是使用 `_kbhit()` 與 `_getch()` 底層非阻塞偵測技術，在背景以每秒 50 次（50 FPS）的高頻率原地重繪，實現流暢的即時計時。

我們引入高精度單調時鐘 `steady_clock` 來精準測量時間差，不受系統時間調整干擾。搭配游標隱形控制，徹底消除高頻刷新帶來的閃爍問題，提供穩定的視覺體驗。

計時功能

之所以將遊戲的渲染刷新率設定在 50 FPS（每 20 毫秒刷新一次），是基於軟體效能優化與作業系統相容性的考量。

在純文字、數字組成的數獨遊戲畫面中，50 FPS 的流暢度在視覺上已經等同於視窗軟體，人眼已經完全無法察覺到任何時間停頓，再往上提高（如 100 FPS）對文字畫面沒有任何視覺幫助。

```
if (difficulty == 4) {  
    timeLimitInSeconds = 300;  
    maxHints = 0; // 極限模式無提示功能  
}  
else {  
    timeLimitInSeconds = 0;  
    maxHints = 3; // 簡單、普通、困難享有 3 次提示  
}  
  
startTime = std::chrono::steady_clock::now();  
  
string inputBuffer = "";  
string systemMessage = "";  
bool timeOut = false;
```

```
bool inputComplete = false;
while (true) {
    auto currentTime = chrono::steady_clock::now();
    int elapsedSeconds = chrono::duration_cast<chrono::seconds>(currentTime - startTime).count();
    int remainingTime = timeLimitInSeconds - elapsedSeconds;

    if (timeLimitInSeconds > 0 && remainingTime <= 0) {
        timeOut = true;
        break;
    }

    clear();
}
```

```
cout << "===== 隨機生成數獨挑戰賽 =====\n";
```

```
cout << " 目前錯誤次數: " << wrongCount << " / " << maxWrong << "\033[K\n\n";
```

```
if (timeLimitInSeconds > 0) {
```

```
    cout << " [極限挑戰] 剩餘時間: " << remainingTime / 60 << " 分 " << remainingTime % 60 << " 秒\033[K\n";
```

```
}
```

```
else {
```

```
    cout << " [計時模式] 已遊玩時間: " << elapsedSeconds / 60 << " 分 " << elapsedSeconds % 60 << " 秒\033[K\n";
```

```
}
```

```
if (maxHints > 0) {
```

```
    cout << " [提示] 剩餘提示機會: " << (maxHints - hintCount) << " / " << maxHints << " (輸入 0 0 0 取得提示)\033[K\n\n";
```

```
}
```

```
else {
```

```
    cout << " [提示] 極限模式不提供提示功能！\033[K\n\n";
```

```
}
```

```
display();
```

```
if (!systemMessage.empty()) {
    cout << "\n" << systemMessage << "\033[K\n";
}
else {
    cout << "\n\033[K\n";
}

cout << "\n請輸入作答 [列] [行] [數字] (例如: 1 3 4): " << inputBuffer << "\033[K";

// \033[?25h：在畫面的最後一步將游標顯示回來，讓玩家看得到輸入游標的位置
cout << "\033[?25h" << flush;

if (_kbhit()) {
    char ch = _getch();

    if (ch == '\r') {
        if (!inputBuffer.empty()) {
            inputComplete = true;
            break;
        }
    }
    else if (ch == '\b') {
        if (!inputBuffer.empty()) {
            inputBuffer.pop_back();
        }
    }
    else if (ch >= 32 && ch <= 126) {
        inputBuffer += ch;
    }
}

this_thread::sleep_for(chrono::milliseconds(20));
```

```
void Game::clear() {  
    // \033[?25l：隱藏游標（避免在重繪盤面時游標閃爍亂跑）  
    // \033[1;1H：將游標移回左上角  
    cout << "\033[?25l\033[1;1H";  
}
```

判斷對錯

直接和正確答案比對，避免玩家可能要繼續往後填了 10 幾格之後，才會發現整個盤面被卡死無解。

```
bool Sudoku::isSafe(int r, int c, int val) const {  
|   return checkRow(r, val) &&  
|       checkCol(c, val) &&  
|       checkBox(r - r % 3, c - c % 3, val);  
|  
| }  
}
```

```
bool Sudoku::validateAndSet(int r, int c, int val) {  
    if (solution[r][c] == val) {  
        grid[r][c] = val;  
        return true;  
    }  
    return false;  
}
```

工作分配

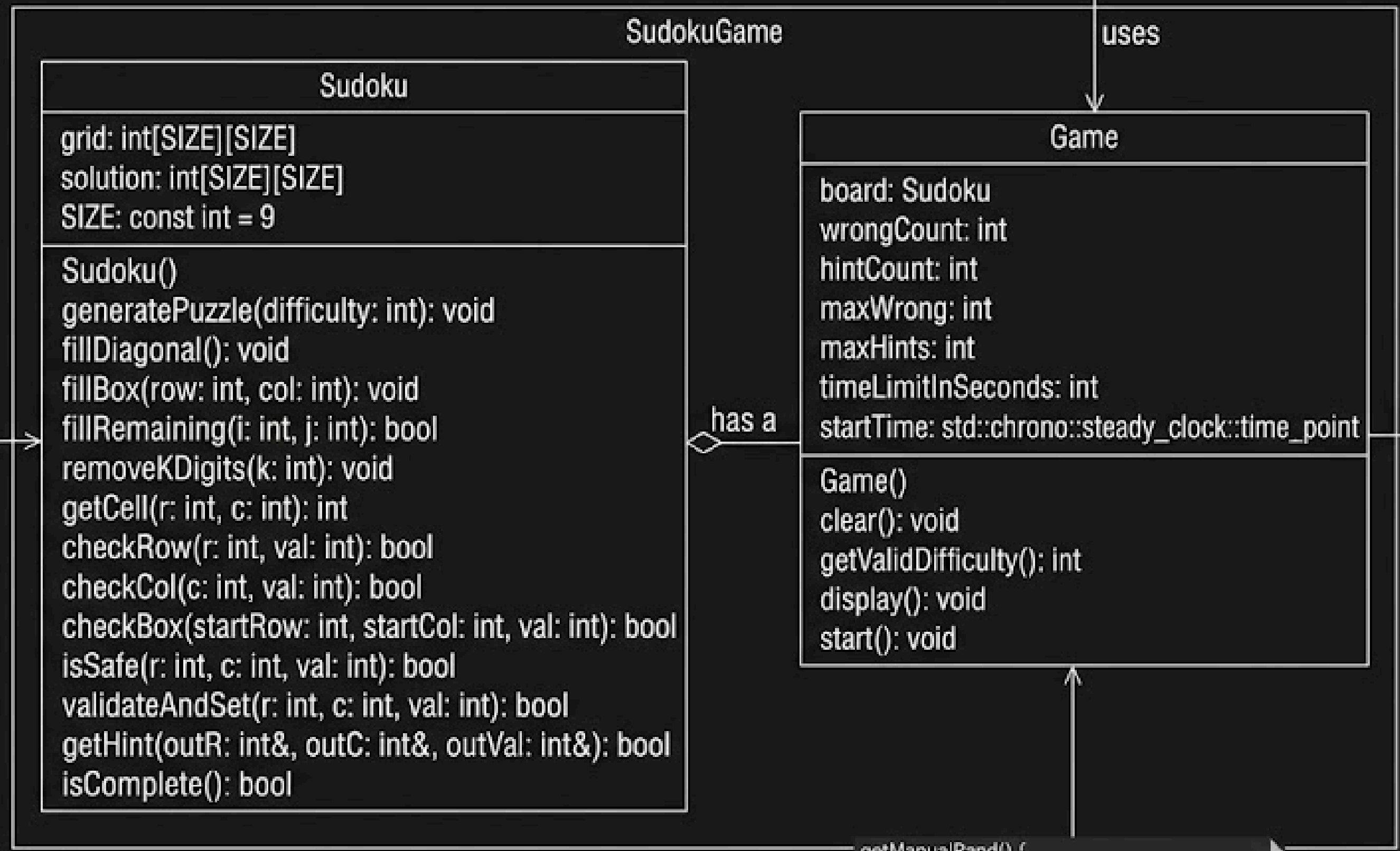
邱岱瑋:程式、簡報製作

鍾錦沛:程式、簡報製作

```
// main.cpp:and
class getManualRand(difficulty: int) {
    int row = SIZE();
    int manualRand(const int n);
    int getManualRand();

    for (int row: int, i=0; i< n; i++) {
        ...
        solutionview: int (i + j) > 9;
    }

    void loadited(difficulty: int) = Set();
    return main();
}
```



```
getManualRand() {
    int successcin(difficulty, startCol, val, vot);
    const r = 0; getManualRand(i, - 1);
    return (int, vell);
}
```

